

Week 6 – Apache Spark Assignment

Name : Siga Nithin

College : CVR College Of Engineering

StudentId : CT_CSI_DE_649

Objective

The objective of this assignment is to understand Spark architecture and perform efficient data processing using transformations, filtering, schema handling, optimized file formats, and Spark performance concepts.

Dataset Description

For this assignment I created my own dataset instead of downloading one, so that I could control what goes into it. The dataset (orders_dataset_week6.csv) has around 2,000 records with order, product, pricing, category, status, priority, region and user information. It has 13 columns in total: order_id, user_id, product_id, category, product_name, base_price, price, amount, quantity, region, priority, status and order_date.

A few things were added on purpose so that null handling, casting and de-duplication could actually be demonstrated:

- Some missing (null) values in user_id
- price stored as a String, even though it is really a number, to practice casting it to Double
- A small number of duplicate rows
- Multiple categories, regions, priorities and order statuses

Assignment WorkFlow

The work was done in the following order:

- Created a sample orders dataset with realistic columns and some intentional nulls/duplicates
- Loaded the CSV into Spark using inferSchema
- Explored the dataset using printSchema(), columns, count() and describe()
- Defined and applied a custom schema instead of relying only on inferSchema
- Renamed a column and cast the price column from String to Double
- Added a new calculated column, final_price
- Filtered the data using single, AND and OR conditions
- Checked and handled null values in user_id
- Removed duplicate rows
- Compared transformations and actions using groupBy + orderBy + show()
- Saved the data as both CSV and Parquet, and compared the two
- Read the Parquet file back, filtered out null user_id rows, and wrote the final output as CSV
- Completed an end-to-end read → transform → filter → write Spark pipeline

Thoery Questions (1-15) :

Q1. What is PySpark and why is it used for big data processing?

PySpark is basically the Python interface to Apache Spark. It lets Spark jobs be written in Python while the actual processing still runs on Spark's distributed engine underneath. The reason it's used for big data is that it splits a large dataset into partitions and processes them in parallel across multiple machines, instead of trying to load everything into the memory of one machine the way pandas would.

Q2. Explain the Spark architecture (Driver, Executors, Cluster Manager).

Every Spark application runs with three main parts working together. The Driver is the program that starts the SparkSession, builds the DAG for whatever job is running, and hands out tasks to be done. The Cluster Manager is responsible for allocating the actual machines/resources the job needs. The Executors then run on those worker nodes and do the real processing - reading their partition of data, running the tasks, and sending the results back to the driver.

Q3. How is a CSV file read in PySpark with header and schema inference?

```
spark.read.csv(path, header=True, inferSchema=True)
```

header=True tells Spark that the first row is column names rather than data, and inferSchema=True makes Spark scan through the file once to guess a datatype for each column on its own. It's an easy way to get a usable DataFrame without writing out a schema by hand, which is why it was used early on for exploring the dataset (notebook, Step 3).

Q4. What is the difference between CSV and Parquet file formats?

CSV keeps data row by row as plain text, so it's easy to open and read but larger on disk, and Spark has to go through the whole row even if only one column is actually needed. Parquet instead stores data column by column and compresses it, so Spark can pick out just the columns it needs and skip the rest. Parquet files also carry their schema along with them, while a CSV file doesn't store any type information at all. This difference showed up clearly in the notebook, where the Parquet folder ended up noticeably smaller than the CSV folder for the exact same data.

Q5. How are rows filtered based on a condition (category = Electronics)?

```
df.filter(col("category") == "Electronics")
```

This just keeps the rows where the category column matches "Electronics" and drops everything else, which is the same filter used in the notebook to pull out electronics orders.

Q6. How is a column renamed and its datatype cast in PySpark?

```
df.withColumnRenamed("product_name", "item_name")  
df.withColumn("price", col("price").cast(DoubleType()))
```

withColumnRenamed() just changes a column's name without touching its values, while withColumn() together with .cast() rebuilds a column with a new datatype - here turning price from a String into a proper Double so it can be used in calculations later on.

Q7. What is the difference between Client mode and Cluster mode?

In Client mode, the driver runs on whichever machine submitted the job, so that machine needs to stay connected the whole time for the job to finish. In Cluster mode, the driver itself runs inside the cluster, which makes it more reliable for long production jobs since it isn't tied to the machine that kicked things off. A Databricks notebook session is really a managed version of this same idea, where Databricks handles where the driver actually lives.

Q8. How are multiple filter conditions (AND) applied in PySpark?

```
df.filter((col("status") == "Completed") & (col("amount") > 1000))
```

Wrapping each condition in brackets and joining them with & means both conditions have to be true for a row to be kept - here, only Completed orders with an amount above 1000 pass through.

Q9. What is predicate pushdown and how does Parquet support it?

Predicate pushdown is when the filter condition gets pushed down to the storage layer instead of Spark first loading everything into memory and filtering afterwards. Parquet makes this possible because it keeps min/max statistics for each column at the row-group level, so Spark can look at those stats and skip whole row groups that couldn't possibly match the filter, without ever reading them. This is part of why filtering on Parquet ends up faster than filtering the same condition on CSV.

Q10. How is a new derived column added using withColumn()?

```
df.withColumn("final_price", col("base_price") * 1.18)
```

withColumn() adds a new column, final_price, calculated by multiplying the existing base_price column by 1.18 to work in 18% tax.

Q11. What is the difference between Transformations and Actions in Spark?

Transformations - things like select(), filter(), withColumn() or groupBy() - don't actually do anything by themselves, they just get added as a step to Spark's plan. Actions - show(), count(), collect() and so on - are what actually make Spark go and execute that plan to produce a real result. In the notebook this shows up in the groupBy/orderBy step, where nothing happens until the final show() call runs.

Q12. How is a Parquet file read, filtered, and written back out as CSV?

```
spark.read.parquet(path)
    .filter(col("user_id").isNotNull())
    .write.csv(output_path)
```

Reading the Parquet file back gives the same DataFrame that was saved earlier, filtering it down to rows with a non-null user_id cleans it up, and writing it out as CSV completes the read-transform-filter-write pipeline for the final output.

Q13. What is Lazy Evaluation in Spark?

Spark doesn't run a transformation the instant it's written - it just records it as a step in a lineage graph (DAG) of what needs to happen eventually. It's only once an action gets called

that Spark actually looks at that whole chain, works out the most efficient way to run it, and executes it. That's the reason nothing seems to "happen" until `show()` or `count()` shows up.

Q14. How is an OR condition applied while filtering (region = North OR priority = High)?

```
df.filter((col("region") == "North") | (col("priority") == "High"))
```

Using `|` instead of `&` means a row only needs to satisfy one of the two conditions to be kept, so this keeps every row that is either from the North region or marked High priority.

Q15. What are some best practices to avoid performance issues (e.g. avoiding `collect()`)?

`collect()` drags the entire dataset back to the driver, which can easily overwhelm it once the data gets large, so `show()`, `limit()` or `take()` are better choices when the goal is just to preview data. Beyond that, it generally helps to prefer Parquet over CSV, filter and select the columns actually needed before running heavier operations, and avoid running `groupBy` or `join` more times than necessary since each one triggers a shuffle across the cluster.

Learning Outcomes

- Understood Spark Architecture and its components.
- Learned the difference between Transformations and Actions.
- Practiced reading data with schema inference and custom schemas.
- Performed filtering using single, AND, and OR conditions.
- Learned to rename columns and cast datatypes.
- Compared CSV and Parquet storage formats.
- Understood Lazy Evaluation and DAG execution.
- Learned how Predicate Pushdown improves performance.
- Built an end-to-end Spark data processing pipeline.
- Applied Spark best practices for handling large datasets.

Conclusion

This assignment covered both the practical and theoretical sides of working with PySpark. On the practical side, a custom dataset was built and put through a complete pipeline - reading, schema handling, column operations, filtering, null and duplicate handling, and saving in both CSV and Parquet. On the theory side, the Spark architecture, lazy evaluation, fault tolerance, shuffle and predicate pushdown concepts were tied back to the code that was actually written. Overall, this gave a much clearer picture of how Spark works internally and not just how to call its functions.